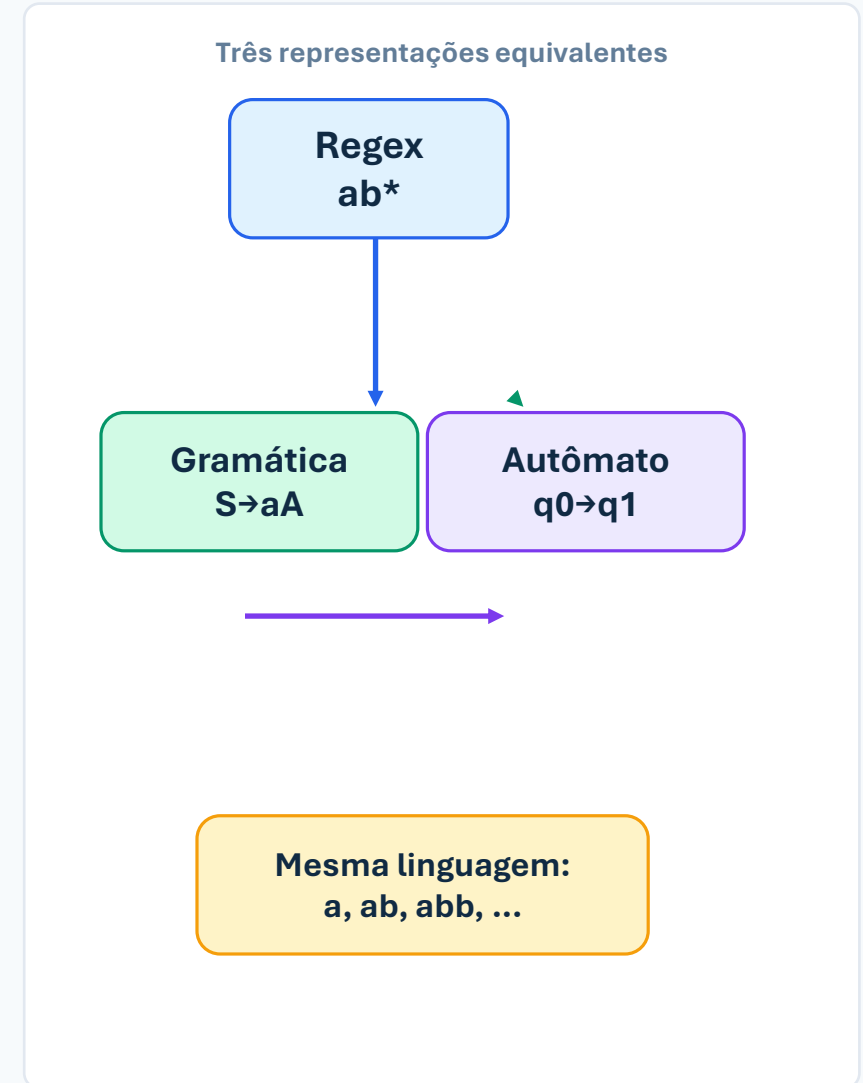


Conversões e minimização de autômatos

Tópico: Visão geral da aula

Nesta aula vamos estudar como expressões regulares, gramáticas regulares e autômatos finitos podem representar a mesma linguagem regular. Também veremos por que converter entre modelos ajuda a projetar analisadores léxicos, testar padrões e simplificar máquinas por meio da minimização de autômatos.

[JFLAP oficial: ferramenta gratuita para simulação](#)



Relação com o plano da disciplina

Tópico: Conteúdo previsto

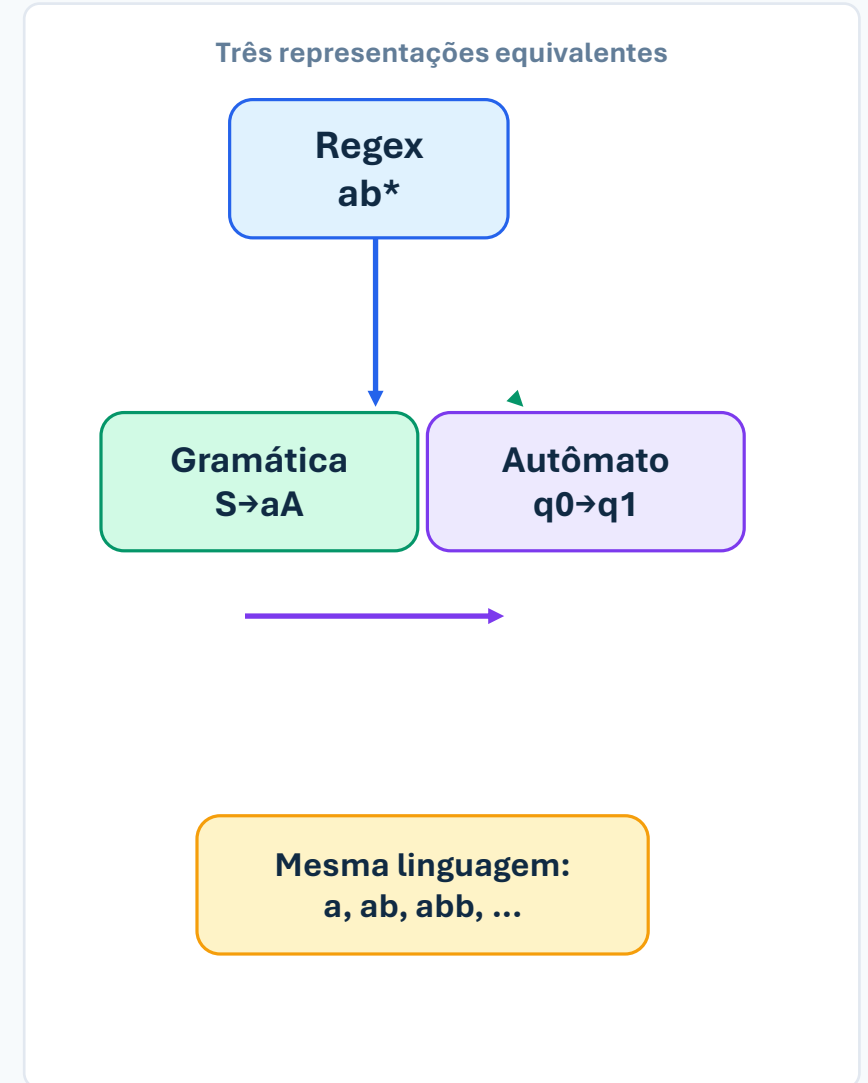
O tema corresponde ao estudo de conversões entre expressões regulares, gramáticas regulares e autômatos, incluindo minimização. Esse conteúdo aprofunda a parte de linguagens regulares e prepara a transição para análise léxica, onde padrões formais são transformados em reconhecimento automático de tokens.



Por que converter representações?

Tópico: Motivação

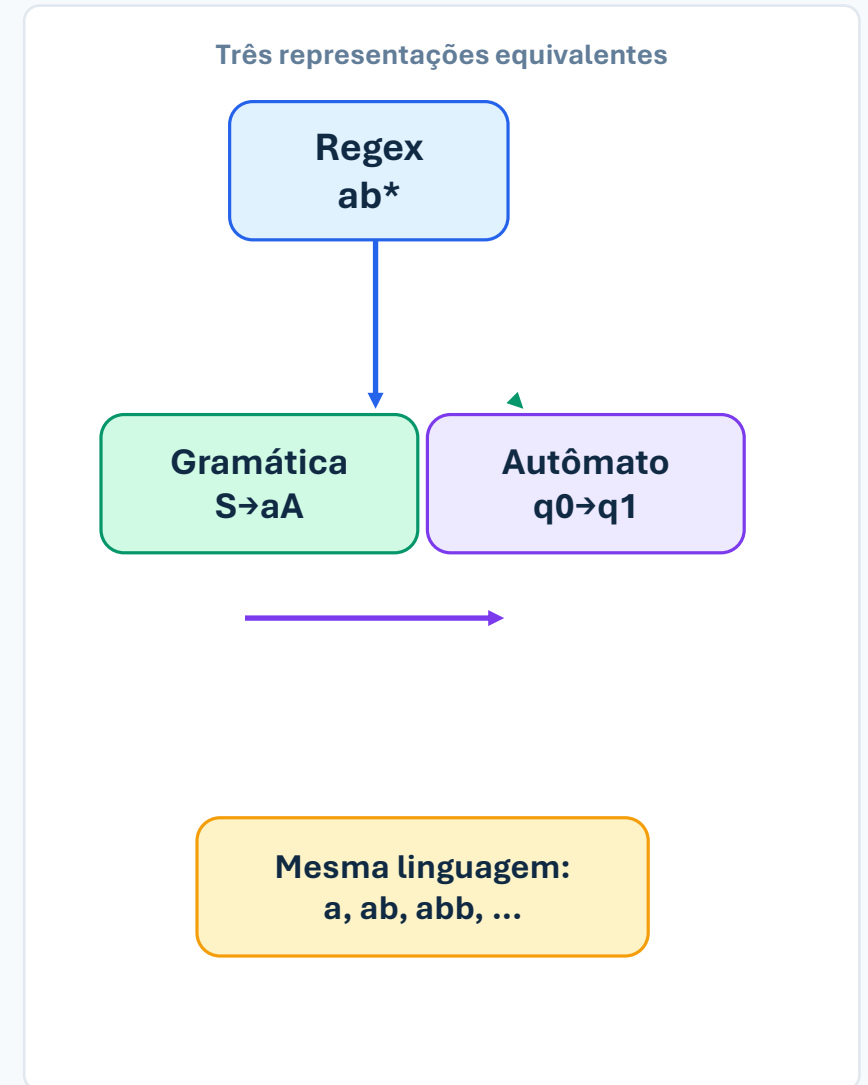
Cada representação tem uma vantagem didática e prática. Expressões regulares são compactas para escrever padrões; gramáticas evidenciam geração de palavras; autômatos mostram execução passo a passo. Converter permite escolher a forma mais adequada para explicar, implementar, testar ou otimizar uma linguagem regular.



Linguagem regular como ponto comum

Tópico: Equivalência conceitual

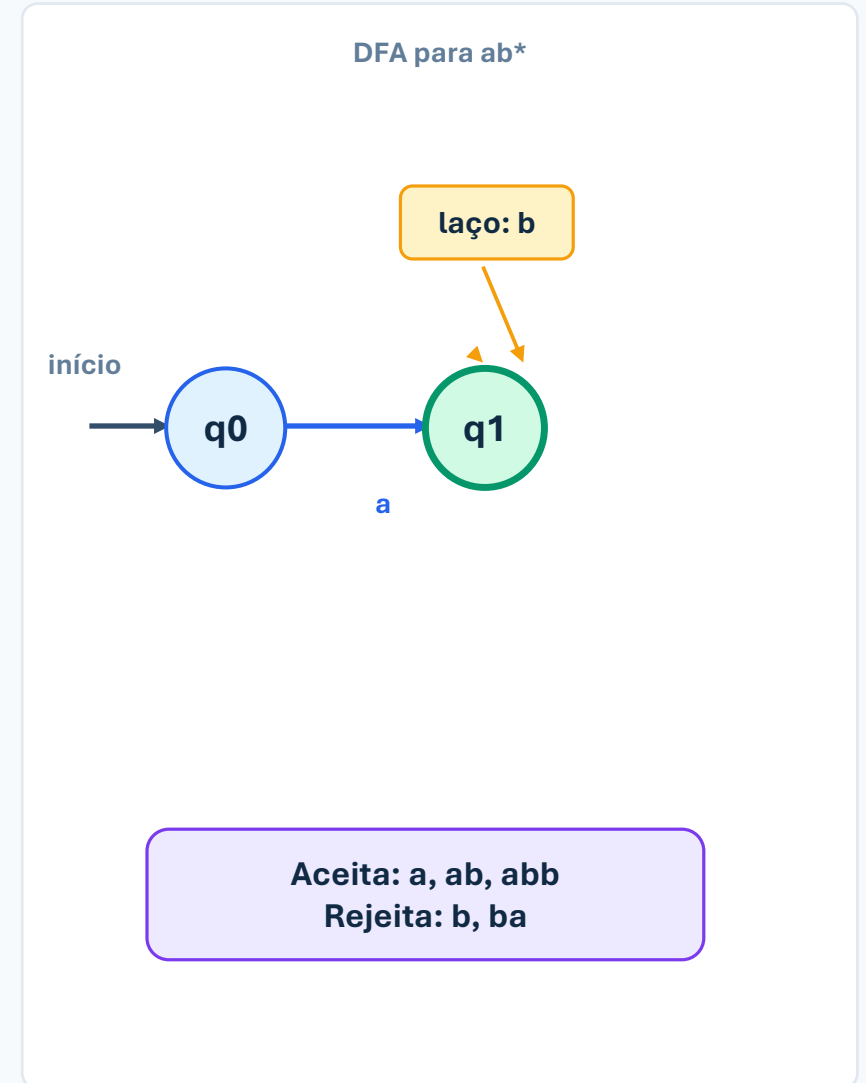
Uma linguagem regular é aquela que pode ser descrita por uma expressão regular, gerada por uma gramática regular ou reconhecida por um autômato finito. Quando convertemos entre essas formas, não mudamos a linguagem; mudamos apenas a maneira de representá-la e analisá-la.



Revisão dos elementos formais

Tópico: Base necessária

Antes das conversões, precisamos recordar alfabeto, palavra, linguagem, estado, transição, estado inicial e estado final. O alfabeto define os símbolos possíveis; a palavra é uma sequência desses símbolos; a linguagem é o conjunto de palavras que desejamos aceitar ou rejeitar.



Conversão: expressão regular para NFA

Tópico: Ideia geral

A conversão de uma expressão regular para um NFA normalmente é feita montando pequenas partes. Cada operador regular possui uma construção: símbolo, união, concatenação e repetição. O resultado final é um autômato não determinístico que reconhece exatamente as palavras descritas pela expressão.

Construção: símbolo



**a reconhece
exatamente “a”**

Construção básica: símbolo

Tópico: Regex → NFA

Para reconhecer um símbolo simples, como a, criamos um estado inicial, um estado final e uma transição rotulada com esse símbolo. Essa pequena máquina aceita apenas a palavra formada por a. Construções maiores são montadas ligando várias máquinas pequenas desse tipo.

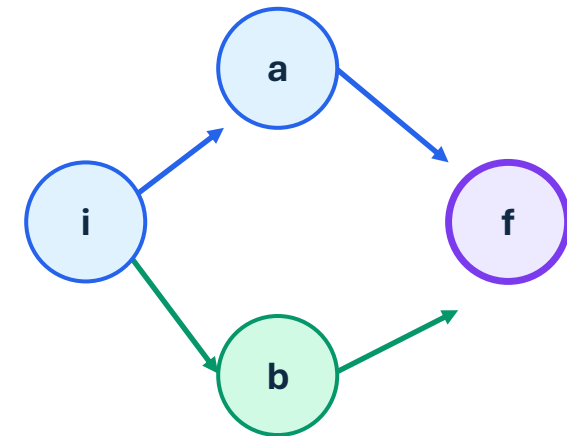


**a reconhece
exatamente "a"**

Construção de união

Tópico: Regex → NFA

Na união, como em $a|b$, o autômato pode escolher entre dois caminhos. Essa escolha é representada por transições vazias, que permitem sair do estado inicial para uma das alternativas sem consumir símbolo. A palavra é aceita se pelo menos um caminho chegar ao estado final.



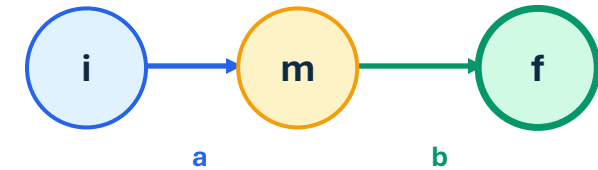
$a | b$:
escolha de caminho
com transições ϵ

Construção de concatenação

Tópico: Regex → NFA

Na concatenação, como em `ab`, a máquina precisa reconhecer uma parte depois da outra. O estado final da primeira parte é conectado ao início da segunda. Assim, a palavra só é aceita quando a sequência completa ocorre na ordem determinada pela expressão regular.

Construção: concatenação

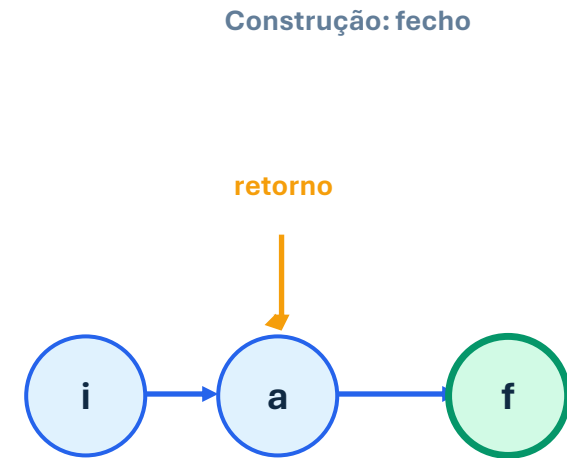


ab:
executa a, depois b

Construção do fecho de Kleene

Tópico: Regex → NFA

O fecho de Kleene, como em a^* , permite zero ou mais repetições. Por isso, a construção precisa aceitar a palavra vazia e também permitir retornar para repetir o padrão. Esse operador é muito comum em identificadores, números, espaços em branco e listas de tokens.

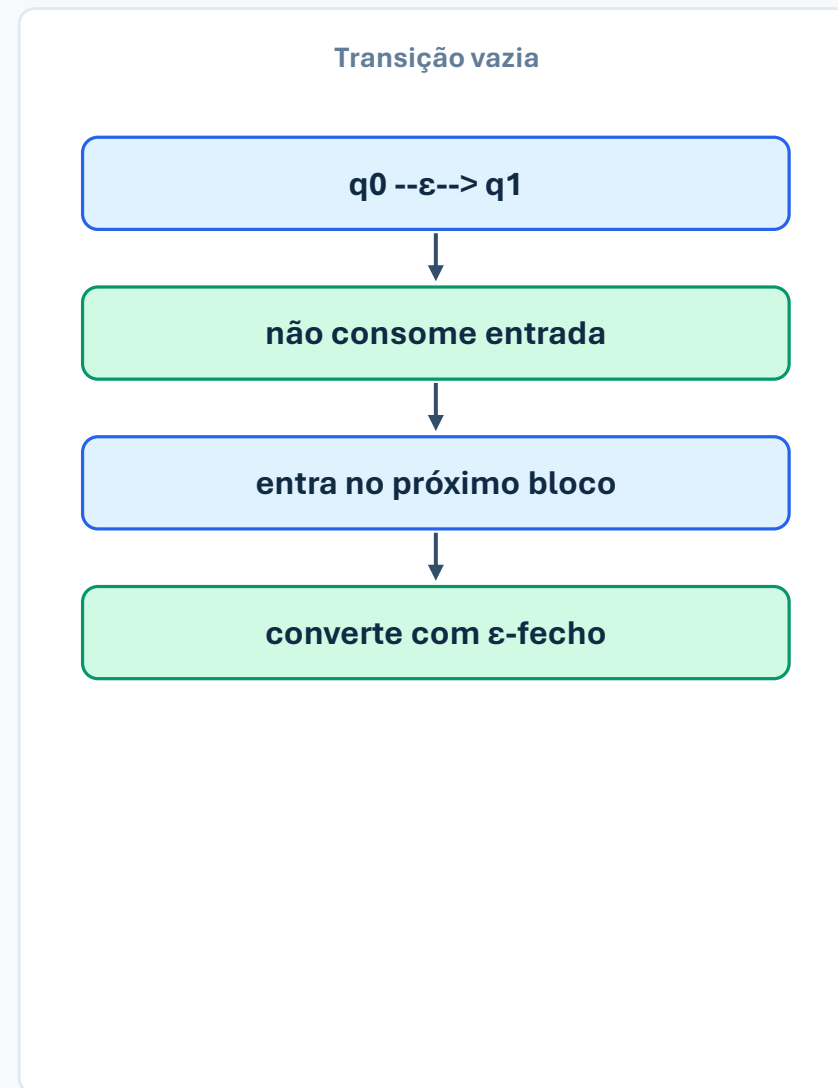


a^* usa ϵ para aceitar vazio e repetir.

Transições vazias na conversão

Tópico: Uso de ϵ ou λ

Transições vazias são essenciais para simplificar a construção de NFAs. Elas permitem mudar de estado sem consumir entrada, conectando partes de uma expressão regular. Embora facilitem o desenho do NFA, precisam ser tratadas cuidadosamente quando convertemos o NFA para um DFA.

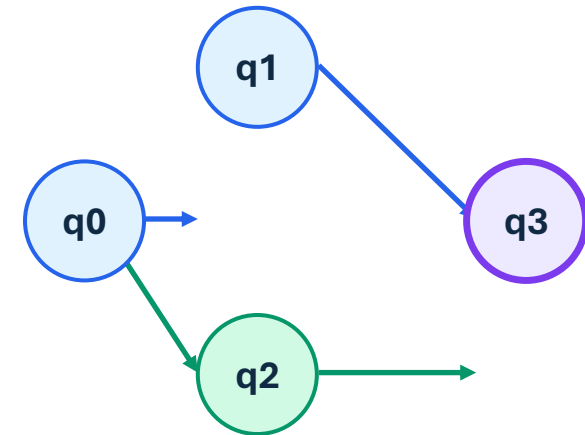


Exemplo: $a(b|c)^*$

Tópico: Montagem por partes

A expressão $a(b|c)^*$ aceita palavras que começam com a e depois possuem qualquer quantidade de b ou c. Podemos construir um NFA iniciando com a transição a e, em seguida, ligando uma repetição com duas alternativas: b ou c.

NFA com caminhos paralelos



ramos: $a \rightarrow b$ ou $a \rightarrow c$
Uma entrada pode ter mais de um caminho.

Testando o NFA construído

Tópico: Validação por exemplos

Depois de construir o NFA, devemos testar palavras aceitas e rejeitadas. Esse teste não substitui a prova formal, mas ajuda a encontrar erros de desenho, como estados finais esquecidos, transições faltantes ou caminhos que aceitam cadeias maiores do que a linguagem desejada.

Testes de aceitação

Estado	a	b
a	OK	OK
ab	OK	OK
acb	OK	OK
b	NÃO	NÃO
ba	NÃO	NÃO

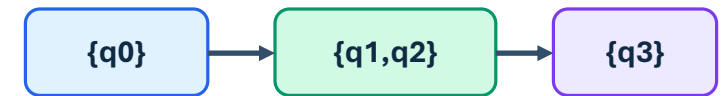
**Tabela ajuda a conferir
se a conversão está correta.**

Conversão: NFA para DFA

Tópico: Construção por subconjuntos

Todo NFA possui um DFA equivalente. A conversão usa subconjuntos de estados: cada estado do DFA representa um conjunto de estados em que o NFA poderia estar após ler uma parte da entrada. Assim, vários caminhos possíveis são organizados em um único caminho determinístico.

Construção por subconjuntos



**Estado DFA =
conjunto de estados NFA**

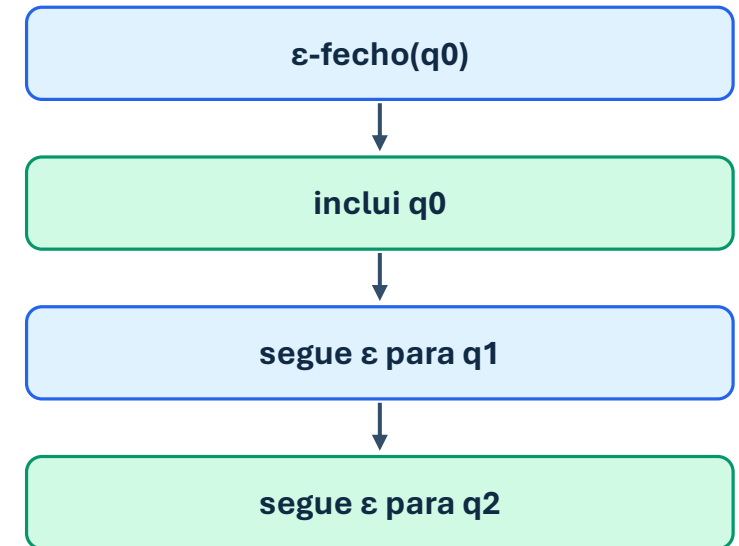
**Se contém estado final
do NFA, é final no DFA.**

O que é ϵ -fecho?

Tópico: Preparação para subconjuntos

O ϵ -fecho de um estado é o conjunto de estados alcançáveis usando apenas transições vazias. Ao converter NFA para DFA, primeiro calculamos esses alcances sem consumir símbolo. Depois, para cada símbolo do alfabeto, avançamos e aplicamos novamente o ϵ -fecho.

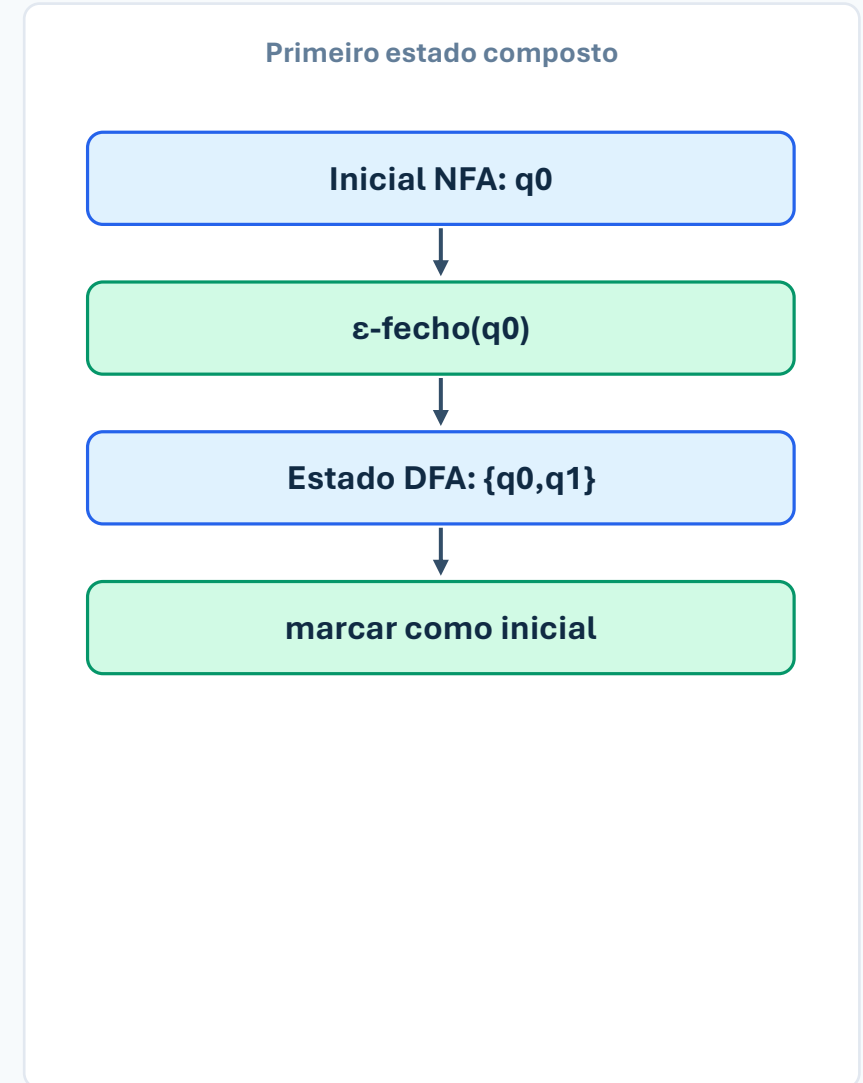
Cálculo do ϵ -fecho



Passo 1: estado inicial do DFA

Tópico: NFA → DFA

O primeiro estado do DFA é formado pelo ε -fecho do estado inicial do NFA. Se o NFA não tiver transições vazias partindo do início, esse conjunto pode conter apenas o próprio estado inicial. Se houver transições vazias, o conjunto inicial será maior.



Passo 2: criar a tabela

Tópico: NFA → DFA

Depois de definir o estado inicial composto, calculamos as transições para cada símbolo do alfabeto. Para cada conjunto atual, aplicamos o símbolo, reunimos os destinos possíveis e calculamos o ε -fecho. Cada novo conjunto encontrado vira um novo estado do DFA.

Tabela de subconjuntos

Estado	a	b
{q0}	{q1,q2}	$\emptyset$
{q1,q2}	{q2}	{q3}
{q3}	$\emptyset$	$\emptyset$
$\emptyset$	$\emptyset$	$\emptyset$

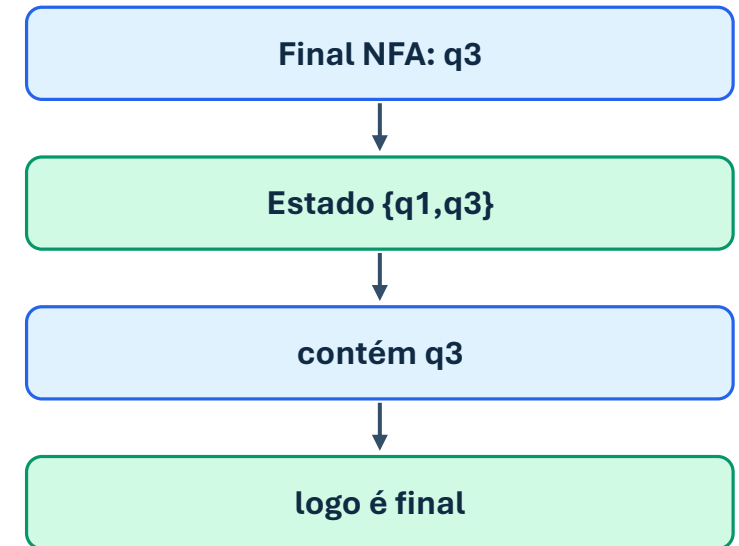
**Tabela ajuda a conferir
se a conversão está correta.**

Passo 3: marcar estados finais

Tópico: NFA → DFA

Um estado do DFA será final se o conjunto que ele representa contém pelo menos um estado final do NFA. Essa regra preserva a aceitação: se alguma possibilidade do NFA terminaria em aceitação, o estado composto correspondente também precisa aceitar.

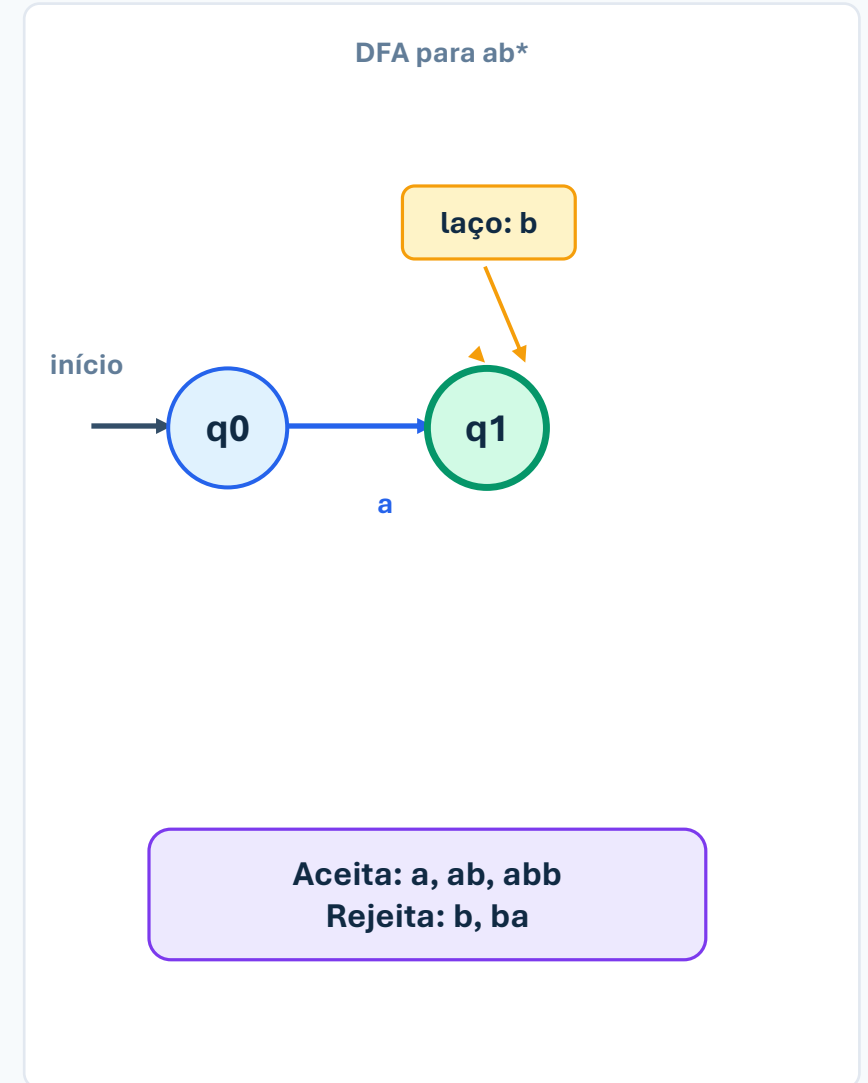
Regra dos estados finais



DFA resultante e linguagem preservada

Tópico: Conferência da equivalência

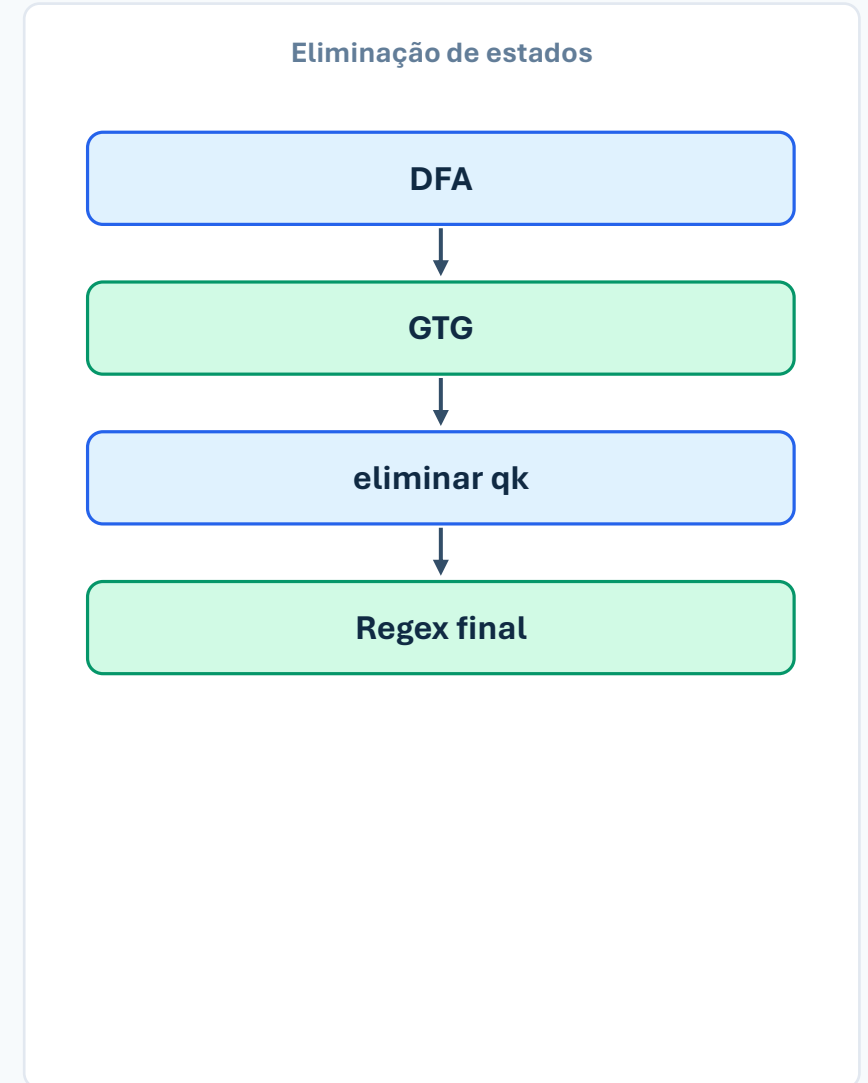
Ao final da conversão, o DFA não possui escolhas múltiplas nem transições vazias. Para cada estado e símbolo, há um próximo estado definido. Mesmo assim, ele reconhece a mesma linguagem do NFA original, aceitando e rejeitando exatamente as mesmas palavras.



Conversão: DFA para expressão regular

Tópico: Eliminação de estados

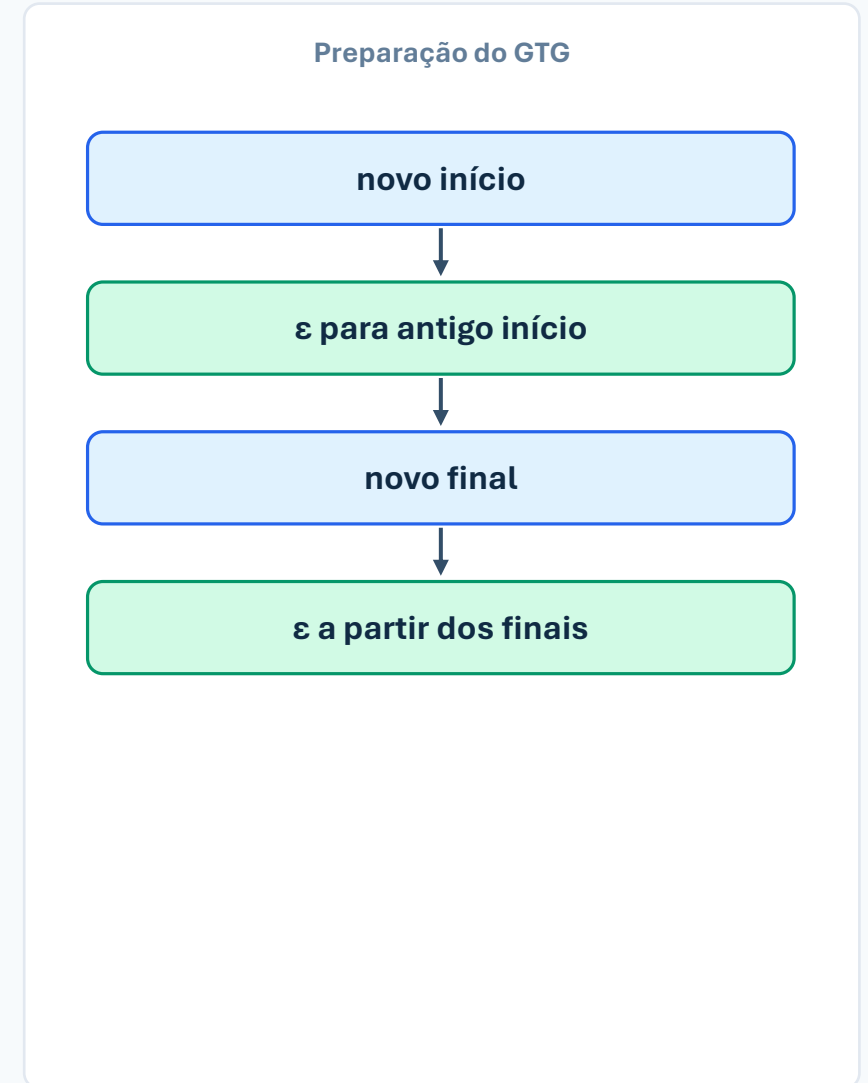
Para obter uma expressão regular a partir de um DFA, podemos usar eliminação de estados. A ideia é transformar caminhos do autômato em expressões nos rótulos das transições e remover estados intermediários, preservando todos os caminhos possíveis entre início e aceitação.



Preparação para eliminar estados

Tópico: DFA → Regex

Antes de eliminar estados, costuma-se criar um único estado inicial e um único estado final, ligados por transições vazias quando necessário. Isso padroniza o processo. Depois, estados intermediários são removidos, e seus caminhos são substituídos por expressões equivalentes.

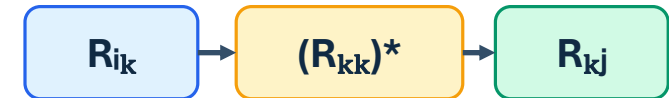


Fórmula intuitiva da eliminação

Tópico: DFA → Regex

Quando removemos um estado k , precisamos considerar caminhos que entram em k , possivelmente dão voltas em k e depois saem. De forma intuitiva, combinamos: entrada, repetição interna e saída. Essa combinação vira uma nova expressão regular entre os estados restantes.

Entrada + repetição + saída



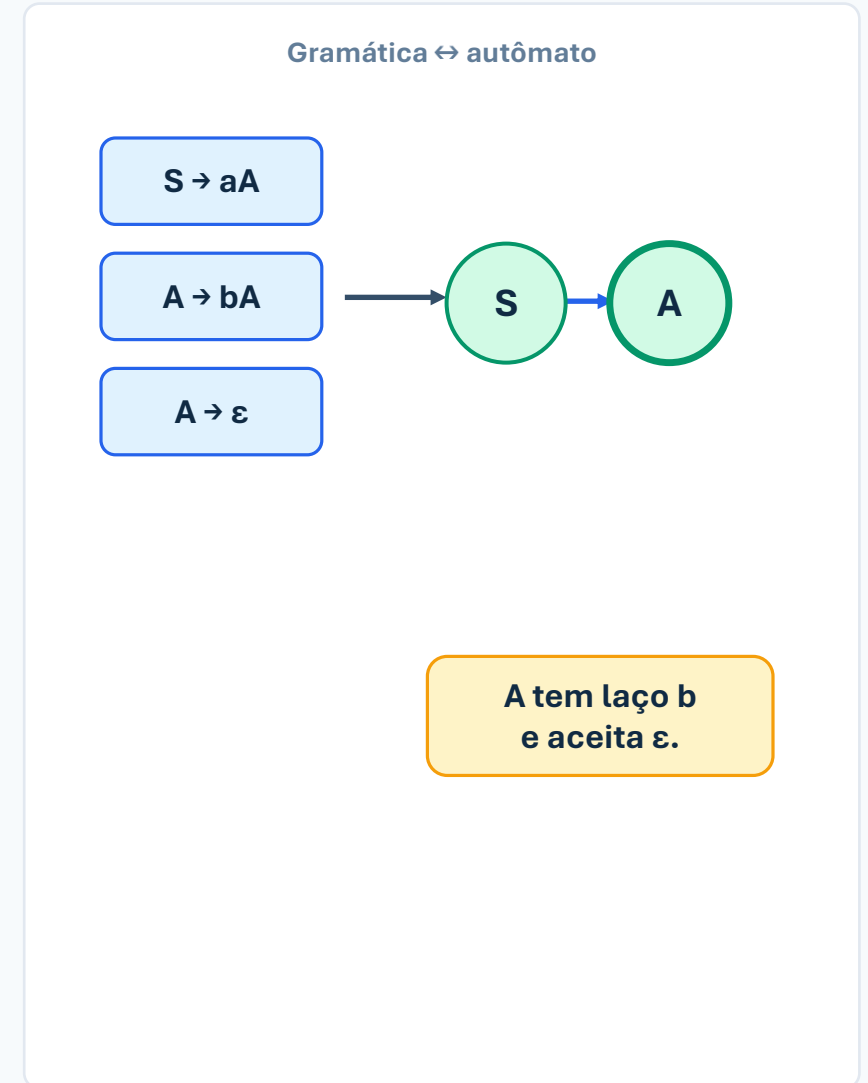
caminho novo:

$R_{ik}(R_{kk})^*R_{kj}$

Gramática regular para autômato

Tópico: Gramática → FA

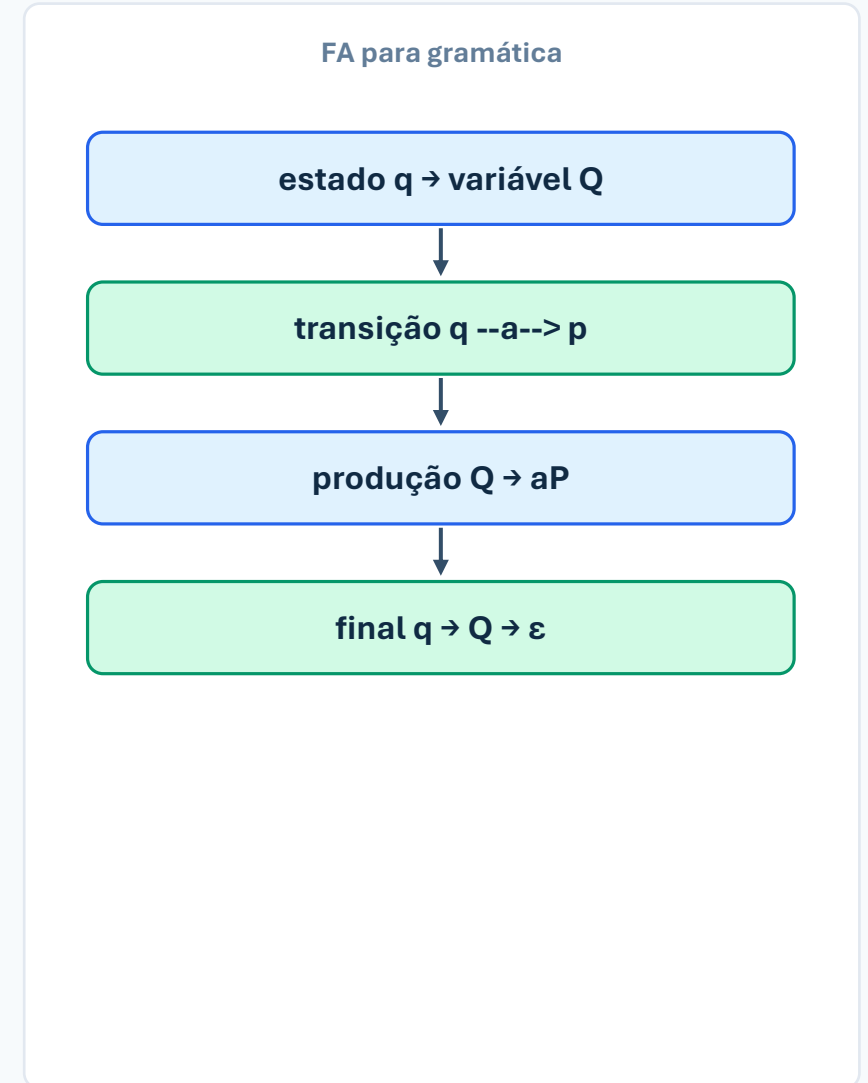
Em uma gramática regular à direita, cada variável pode ser interpretada como um estado. Uma produção como $A \rightarrow aB$ vira uma transição de A para B lendo a . Produções que terminam sem variável indicam chegada a um estado final ou aceitação direta.



Autômato para gramática regular

Tópico: FA → Gramática

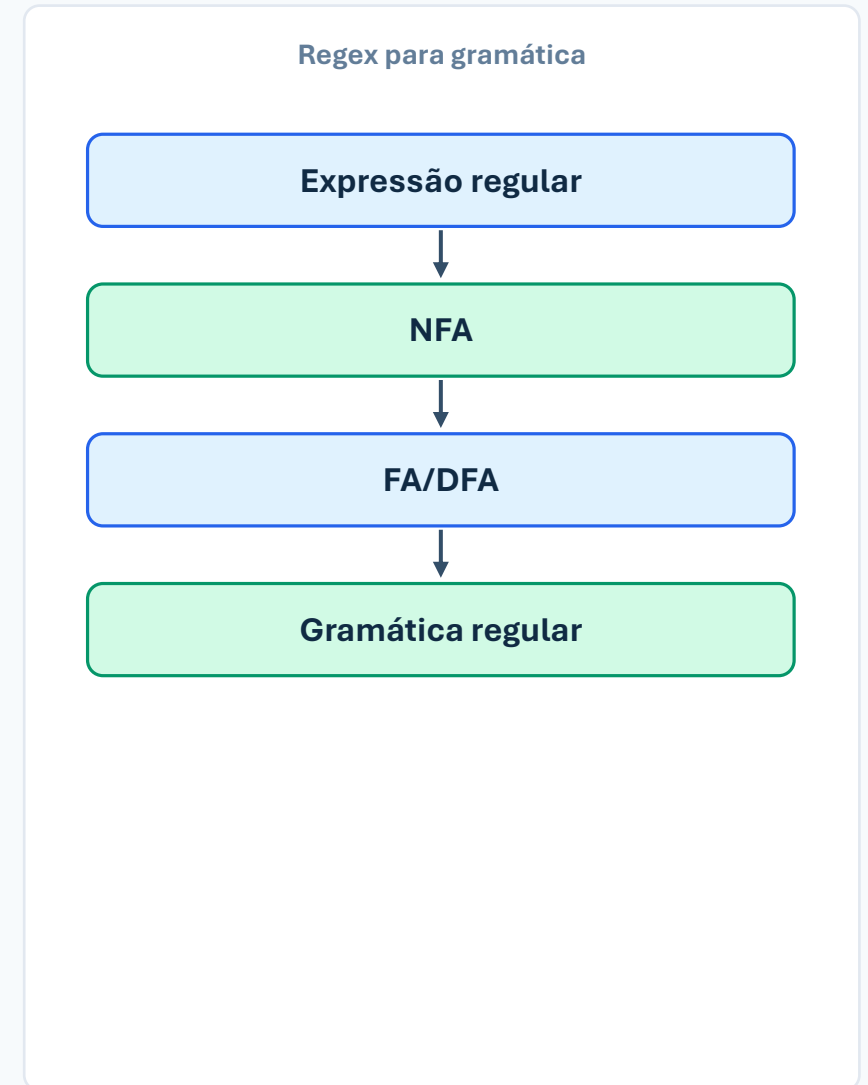
Também podemos converter um autômato em gramática regular. Cada estado vira uma variável. Cada transição $q \rightarrow p$ lendo a vira uma produção $Q \rightarrow aP$. Se um estado é final, adicionamos uma produção para ϵ , permitindo encerrar a derivação naquele ponto.



Regex para gramática regular

Tópico: Conversão por caminho intermediário

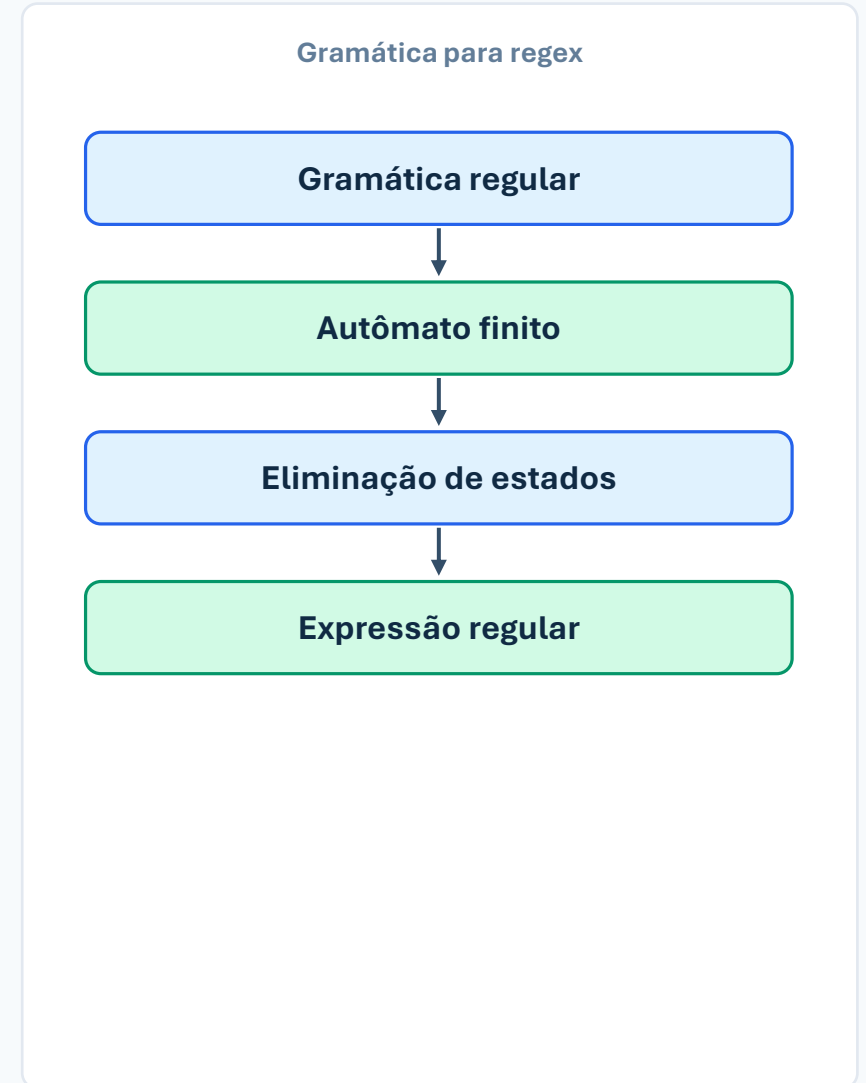
Uma forma prática de obter uma gramática a partir de uma expressão regular é converter primeiro a expressão para um NFA e depois converter o autômato para gramática regular. Esse caminho evita criar regras diretamente a partir da expressão, o que pode ser menos intuitivo.



Gramática para expressão regular

Tópico: Conversão por autômato

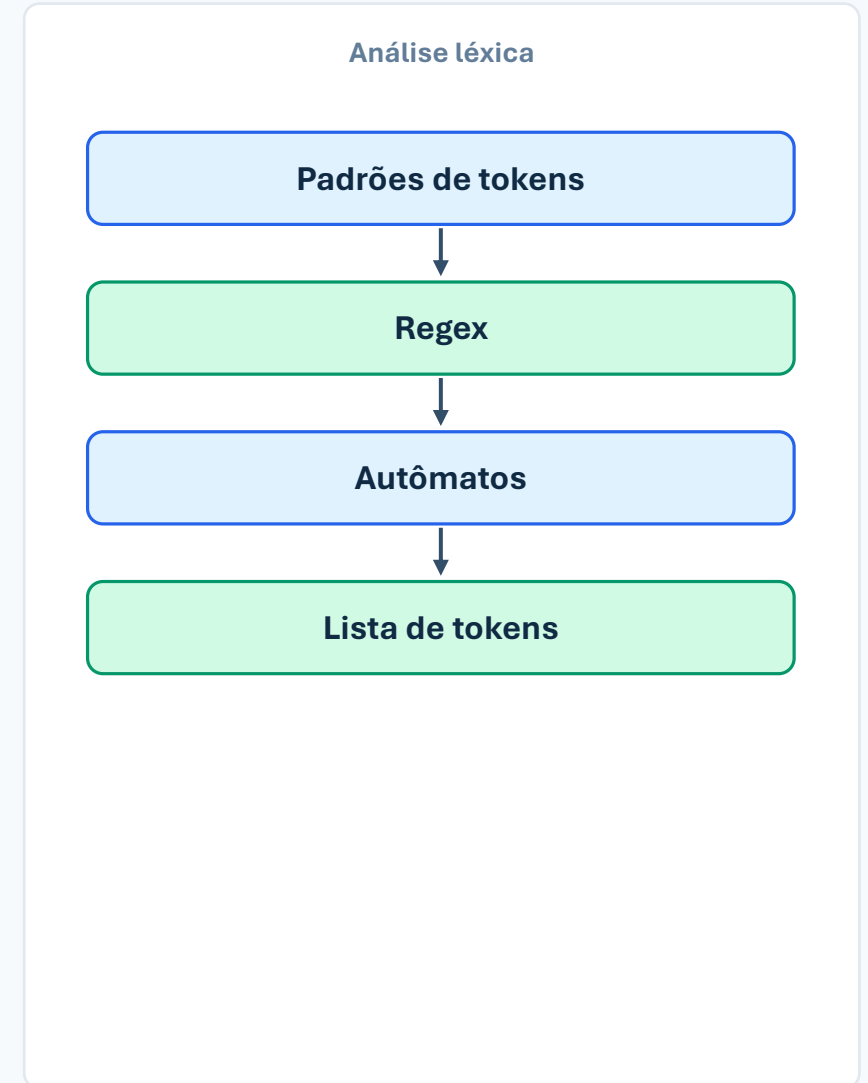
Para obter uma expressão regular a partir de uma gramática regular, podemos converter a gramática em autômato finito e, depois, aplicar eliminação de estados para gerar uma expressão regular. O processo reforça a equivalência entre gerar, reconhecer e descrever linguagens.



Conversões em análise léxica

Tópico: Aplicação em compiladores

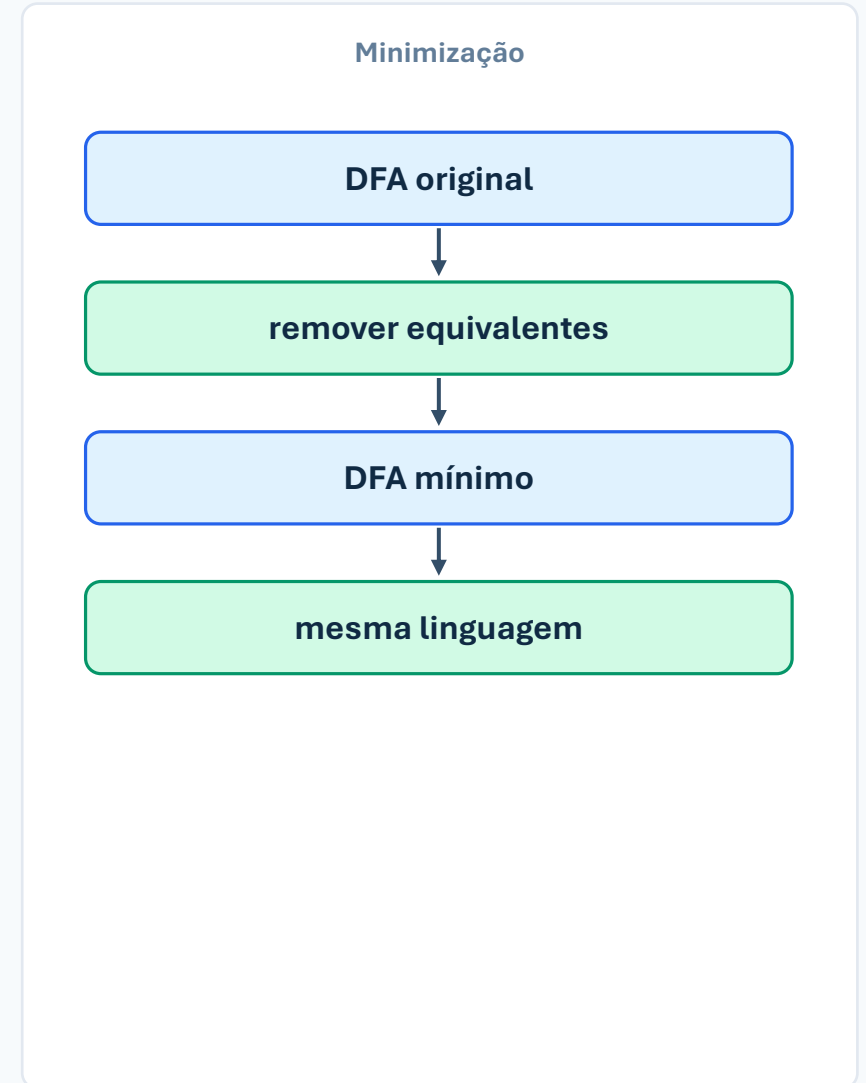
Na análise léxica, padrões de tokens podem ser escritos como expressões regulares. Internamente, esses padrões podem ser convertidos em autômatos para reconhecer lexemas no código-fonte. Essa é uma aplicação direta das conversões estudadas em compiladores reais e didáticos.



Por que minimizar um DFA?

Tópico: Motivação da minimização

Minimizar um DFA significa reduzir a quantidade de estados sem mudar a linguagem reconhecida. Um autômato menor é mais fácil de entender, testar e implementar. Em compiladores, simplificar estruturas pode reduzir complexidade e tornar a execução mais direta.

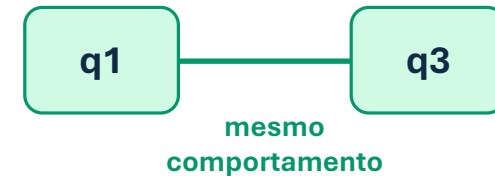


Estados equivalentes

Tópico: Ideia central

Dois estados são equivalentes quando, a partir deles, qualquer continuação possível leva ao mesmo resultado de aceitação ou rejeição. Se não existe palavra capaz de distinguir os dois estados, eles podem ser agrupados sem alterar a linguagem reconhecida pelo autômato.

Estados equivalentes

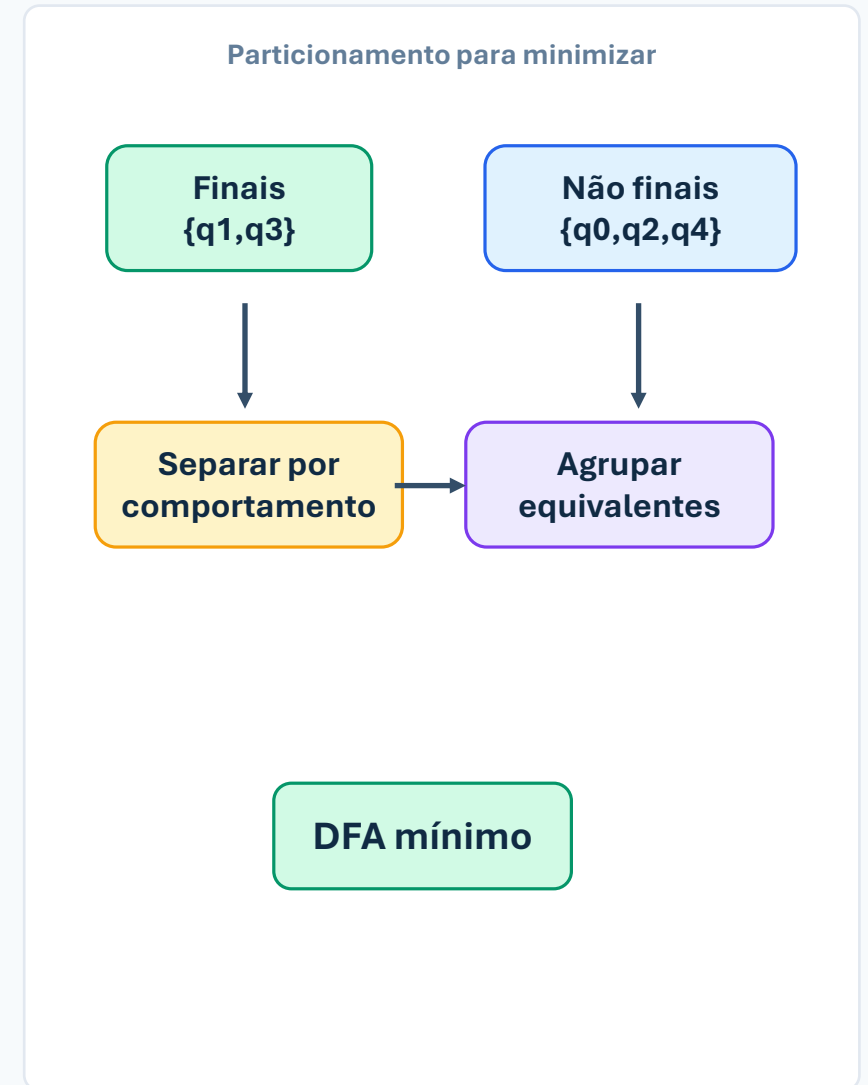


**nenhuma continuação
distingue os dois**

Método de partições

Tópico: Minimização de DFA

Um método comum começa separando estados finais e não finais. Depois, refinamos os grupos conforme o comportamento das transições. Se estados de um mesmo grupo apontam para grupos diferentes com algum símbolo, eles precisam ser separados. O processo termina quando não há novas divisões.



Exemplo de refinamento

Tópico: Minimização passo a passo

Suponha que q_1 e q_3 sejam finais. Inicialmente ficam no mesmo grupo. Se, para o símbolo a , q_1 leva a um grupo diferente de q_3 , então eles são distinguíveis e devem ser separados. Essa verificação é repetida para todos os símbolos.

Refinamento por transições

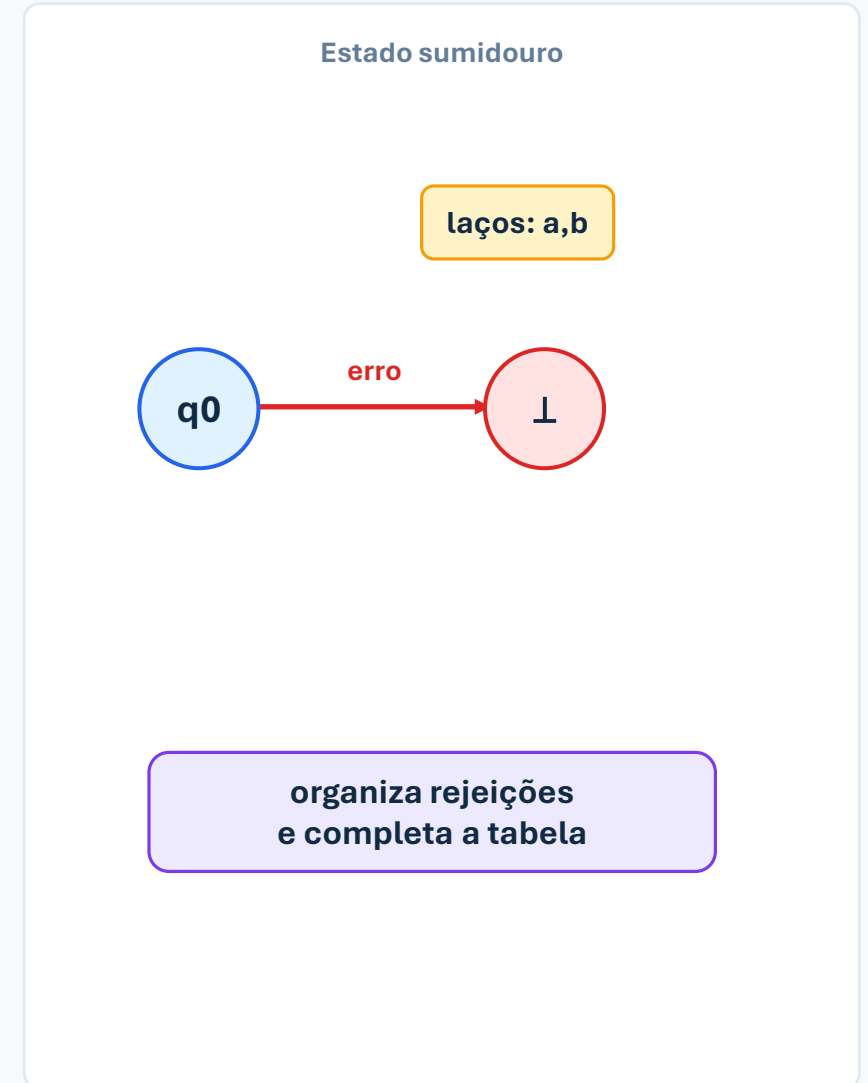
Estado	a	b
q_1	q_2	q_3
q_3	q_4	q_3
q_2	q_2	q_4
q_4	q_4	q_4

**Tabela ajuda a conferir
se a conversão está correta.**

Estado morto ou sumidouro

Tópico: Completude do DFA

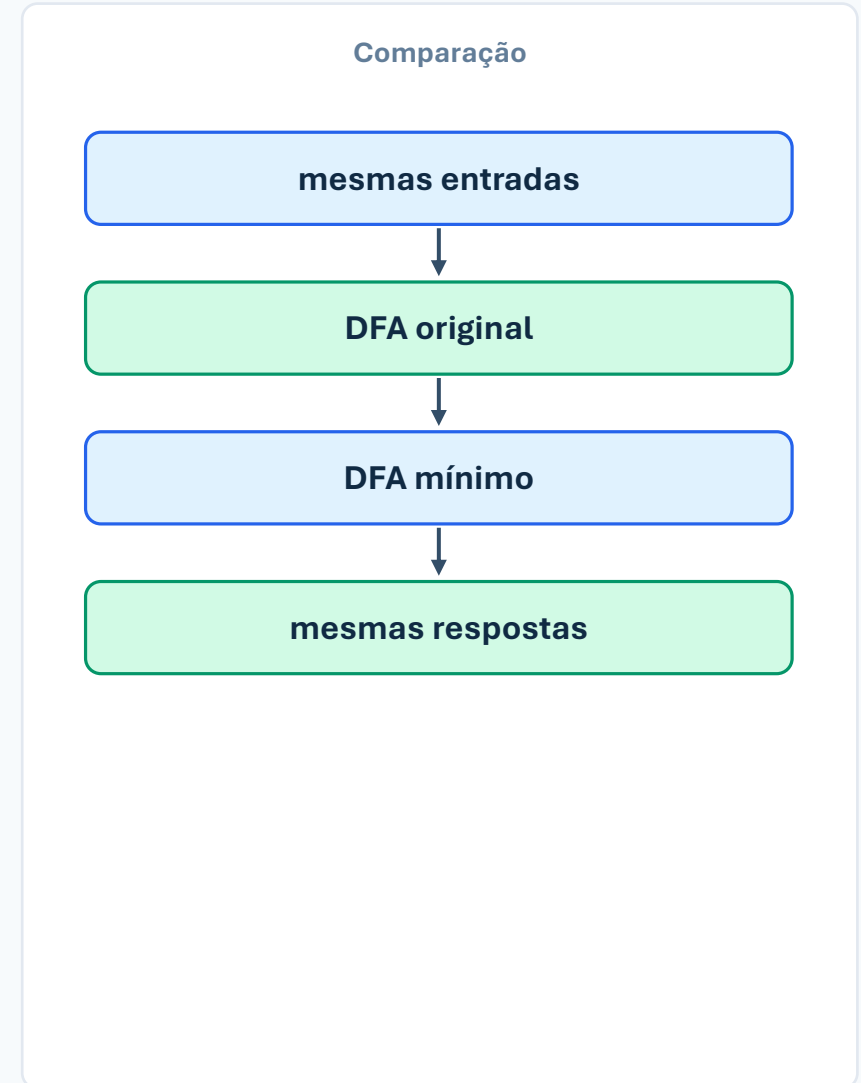
Em muitos exercícios, completamos o DFA adicionando um estado morto, também chamado de sumidouro. Ele recebe entradas que não levam a aceitação e possui laços para todos os símbolos. Isso torna a função de transição total e facilita minimização e implementação.



Conferindo o DFA mínimo

Tópico: Validação pós-minimização

Depois de minimizar, é importante testar o DFA original e o DFA mínimo com as mesmas entradas. As respostas devem coincidir. Se alguma palavra é aceita em um e rejeitada no outro, a minimização alterou a linguagem e precisa ser revisada.



JFLAP como ferramenta prática

Tópico: Aplicação gratuita acadêmica

O JFLAP permite criar autômatos, testar entradas, converter NFA para DFA, minimizar DFA e trabalhar com gramáticas e expressões regulares. Por ser visual e voltado ao ensino, é uma opção adequada para atividades práticas em linguagens formais e compiladores.

[JFLAP: baixar e consultar materiais oficiais](#)

Roteiro JFLAP

1. Abrir JFLAP

2. New

3. Finite Automaton

4. Input > Multiple Run

5. Compare results

Prática 1: regex para autômato

Tópico: Passo a passo no JFLAP

Para converter uma expressão regular no JFLAP, crie uma nova estrutura do tipo Regular Expression, digite o padrão desejado e use a opção de conversão para autômato. Depois, observe o NFA gerado, identifique transições vazias e teste palavras aceitas e rejeitadas.

Regex no JFLAP

1. New > Regular Expression

2. Digitar $a(b+c)^*$

3. Convert to NFA

4. Testar entradas

5. Salvar .jff

Prática 2: NFA para DFA

Tópico: Passo a passo no JFLAP

Depois de criar ou abrir um NFA, use a opção de conversão para DFA. Observe os estados compostos gerados, normalmente escritos como conjuntos. Compare a tabela resultante com a construção por subconjuntos feita manualmente e teste as mesmas palavras no NFA e no DFA.

NFA para DFA

1. Abrir NFA

2. Convert > Convert to DFA

3. Criar estados compostos

4. Conferir transições

5. Testar palavras

Prática 3: minimizar DFA

Tópico: Passo a passo no JFLAP

Com um DFA pronto, use a função de minimização para obter um autômato equivalente com menos estados. Antes de aceitar o resultado, teste entradas no DFA original e no minimizado. A comparação mostra se a linguagem foi preservada e ajuda a entender o agrupamento de estados.

Minimização no JFLAP

1. Abrir DFA

2. Convert > Minimize DFA

3. Comparar estados

4. Testar exemplos

5. Registrar conclusão

Teste rápido com Python

Tópico: Regex em linguagem de programação

Além do JFLAP, é possível testar expressões regulares simples em Python com o módulo `re`. Para comparar com a teoria, use `fullmatch`, pois queremos saber se a palavra inteira pertence à linguagem, e não apenas se um trecho da palavra combina com o padrão.

Python: `re.fullmatch`

```
import re

padrao = r"a(b|c)*"

for w in ["a", "ab", "acb", "b"]:
    print(w,
          bool(re.fullmatch(padrao, w)))
```

Resultado esperado:
True / False

[Documentação Python: módulo re](#)

Atividade guiada em sala

Tópico: Exercício integrado

Proponha aos alunos a linguagem das palavras sobre $\{a,b\}$ que começam com a e terminam com b . Eles devem escrever exemplos aceitos e rejeitados, sugerir uma expressão regular, construir um NFA, converter para DFA e discutir se o DFA pode ser minimizado.

Atividade prática

1. Descrever L em português

2. Listar exemplos

3. Criar regex

4. Construir NFA/DFA

5. Minimizar e testar

Erros comuns nas conversões

Tópico: Cuidados práticos

Os erros mais frequentes são esquecer ε -fecho, marcar estados finais incorretamente, confundir união com concatenação, eliminar estados sem preservar caminhos, minimizar sem completar transições e testar poucas palavras. Pequenos contraexemplos costumam revelar rapidamente uma conversão inadequada.

Erros comuns

1. Esquecer ε -fecho

2. Marcar final errado

3. Confundir * com +

4. Eliminar caminho

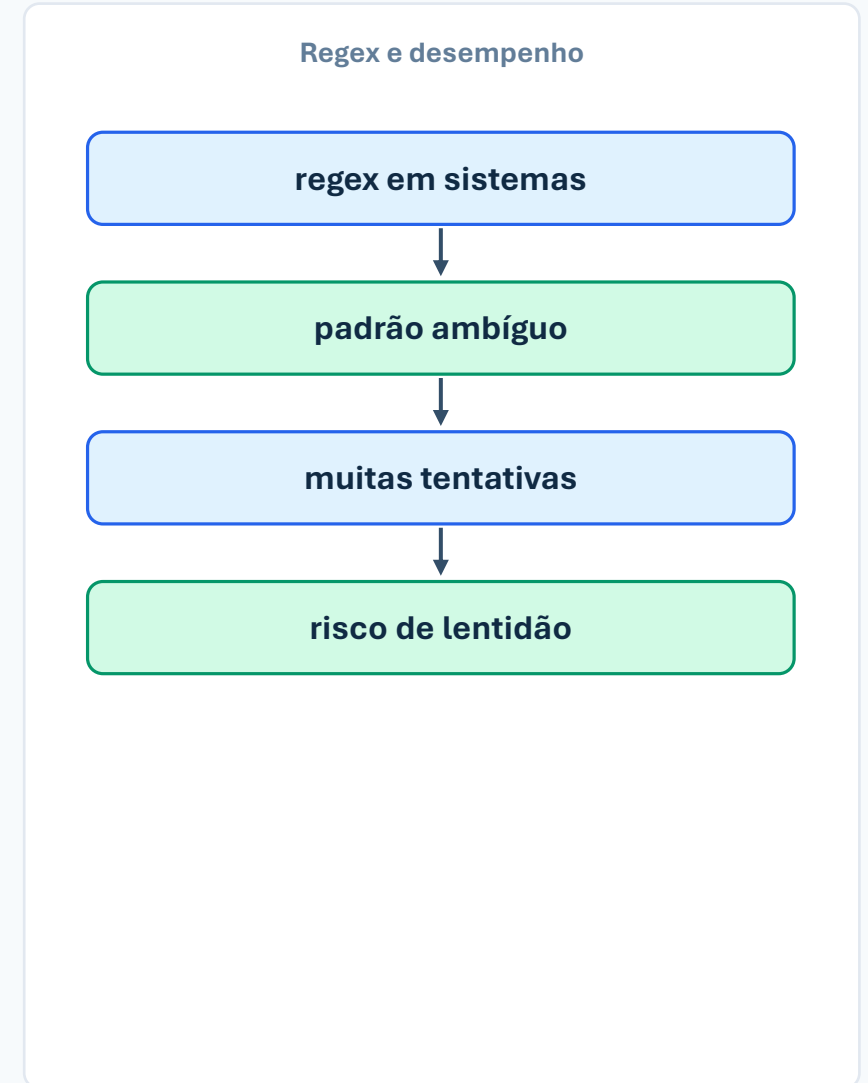
5. Testar pouco

Perspectiva contemporânea

Tópico: Regex, desempenho e segurança

Expressões regulares aparecem em validação de formulários, busca textual, filtros, editores e analisadores. Em aplicações reais, padrões mal construídos podem causar problemas de desempenho. Por isso, entender autômatos ajuda a compreender limites, eficiência e segurança do uso de regex.

Leitura complementar: [OWASP ReDoS](#)



Como estudar conversões

Tópico: Roteiro de aprendizagem

Para estudar bem, resolva cada linguagem em etapas: descreva em português, liste exemplos, escreva a expressão ou gramática, construa o autômato, converta para outra forma, minimize quando for DFA e valide tudo com uma tabela de testes.

Roteiro de estudo

1. Português

2. Exemplos

3. Modelo formal

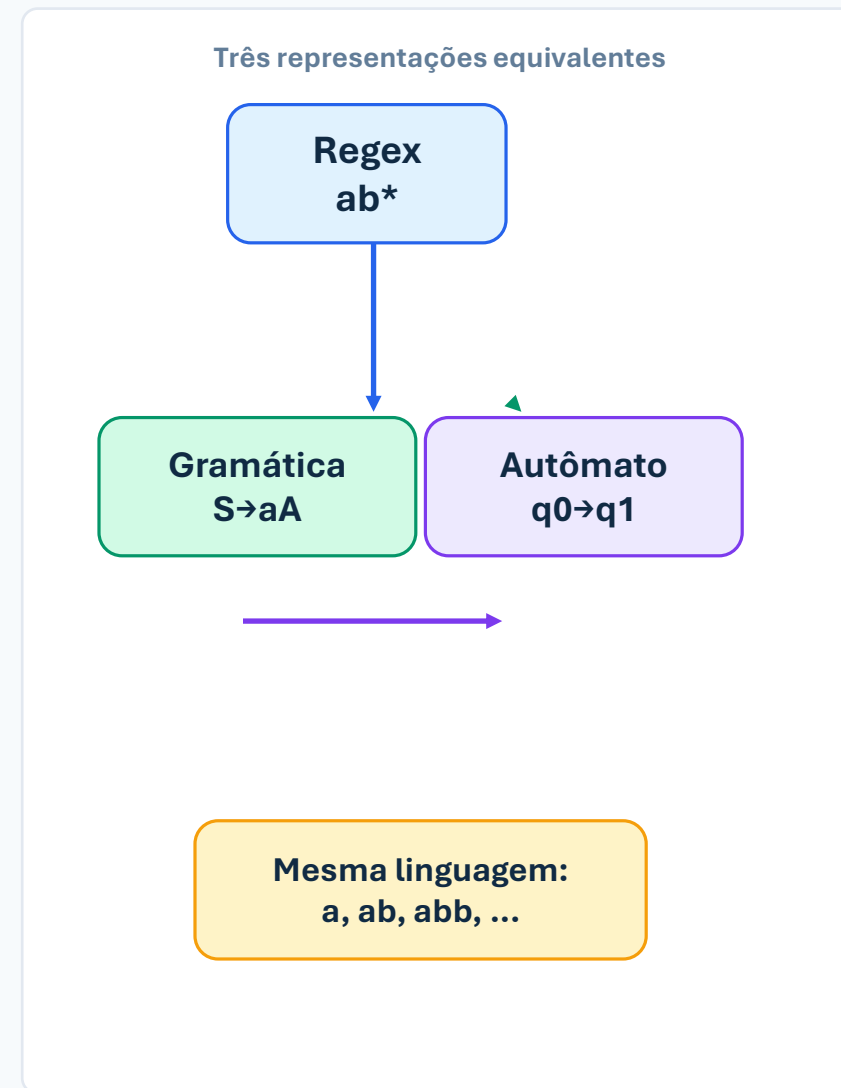
4. Conversão

5. Testes

Síntese da aula

Tópico: O que precisa ficar claro

As conversões mostram que expressões regulares, gramáticas regulares e autômatos finitos têm o mesmo poder para linguagens regulares. A minimização reduz DFAs sem mudar a linguagem. Na prática, esses conceitos apoiam análise léxica, validação de padrões e construção de ferramentas computacionais.



Referências e materiais recomendados

Tópico: Fontes utilizadas

Esta aula utilizou o plano de ensino da disciplina, o material do JFLAP anexado e fontes técnicas abertas para consulta. Recomenda-se que os alunos pratiquem com JFLAP, revisem exemplos de conversão e consultem a documentação do

Python para observar regex em uma linguagem de programação real.

Plano de Ensino de Linguagens Formais e Compiladores 2026/2.

Rodger, S. H.; Finley, T. W. JFLAP – An Interactive Formal Languages and Automata Package, 2005.

JFLAP: <https://www.jflap.org/>

Python re: <https://docs.python.org/3/library/re.html>

OWASP ReDoS: https://owasp.org/www-community/attacks/Regular_expression_Denial_of_Service_-_ReDoS

Stanford CS103: <https://web.stanford.edu/class/cs103/>

Referências

Plano de ensino
2026/2

JFLAP book
Rodger & Finley

Python re
Docs oficiais

OWASP
ReDoS

Stanford CS103
notas abertas

Links completos no quadro à esquerda e nas anotações.